```python
import pandas as pd
import numpy as np

df = pd.read_csv('AeroReach_Insights[1].csv')
print(df.info())
print(df.head())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 11760 entries, 0 to 11759
Data columns (total 17 columns):
 #   Column                                      Non-Null Count
Dtype
---  ------                                      --------------
-----
 0   UserID                                      11760 non-null
int64
 1   Taken_product                               11760 non-null
object
 2   Yearly_avg_view_on_travel_page              11179 non-null
float64
 3   preferred_device                            11707 non-null
object
 4   total_likes_on_outstation_checkin_given     11379 non-null
float64
 5   yearly_avg_Outstation_checkins              11685 non-null
object
 6   member_in_family                            11760 non-null
object
 7   preferred_location_type                     11729 non-null
object
 8   Yearly_avg_comment_on_travel_page           11554 non-null
float64
 9   total_likes_on_outofstation_checkin_received  11760 non-null
int64
 10  week_since_last_outstation_checkin          11760 non-null
int64
 11  following_company_page                      11657 non-null
object
 12  montly_avg_comment_on_company_page          11760 non-null
int64
 13  working_flag                                11760 non-null
object
 14  travelling_network_rating                   11760 non-null
int64
 15  Adult_flag(Age Group)                       11760 non-null
object
 16  Daily_Avg_mins_spend_on_traveling_page      11760 non-null
int64
dtypes: float64(3), int64(6), object(8)
memory usage: 1.5+ MB
```

```
None
    UserID Taken_product  Yearly_avg_view_on_travel_page
preferred_device  \
0  1000001           Yes                            307.0  iOS and
Android
1  1000002            No                            367.0
iOS
2  1000003           Yes                            277.0  iOS and
Android
3  1000004            No                            247.0
iOS
4  1000005            No                            202.0  iOS and
Android

   total_likes_on_outstation_checkin_given
yearly_avg_Outstation_checkins  \
0                                   38570.0
1
1                                    9765.0
1
2                                   48055.0
1
3                                   48720.0
1
4                                   20685.0
1

  member_in_family preferred_location_type
Yearly_avg_comment_on_travel_page  \
0                2         Business Travel
94.0
1                1         Business Travel
61.0
2                2                   Other
92.0
3                4         Business Travel
56.0
4                1         Medical Tourism
40.0

   total_likes_on_outofstation_checkin_received  \
0                                          5993
1                                          5130
2                                          2090
3                                          2909
4                                          3468

   week_since_last_outstation_checkin following_company_page  \
0                                   8                    Yes
1                                   1                     No
```

```
2                                      6                    Yes
3                                      1                    Yes
4                                      9                     No

   montly_avg_comment_on_company_page working_flag
travelling_network_rating  \
0                                  11           No
1
1                                  23          Yes
4
2                                  15           No
2
3                                  11           No
3
4                                  12           No
4

                                Adult_flag(Age Group)  \
0  36 & Above1-36 & Above536 & Above1-36 & Above5...
1              36 & Above1-36 & Above56-36 & Above0
2  36 & Above1-36 & Above536 & Above1-36 & Above5...
3  36 & Above1-36 & Above536 & Above1-36 & Above5...
4              36 & Above1-36 & Above56-36 & Above0

   Daily_Avg_mins_spend_on_traveling_page
0                                      8
1                                     10
2                                      7
3                                      8
4                                      6
```

```python
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler, StandardScaler
from sklearn.cluster import KMeans
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier,
HistGradientBoostingClassifier
from sklearn.metrics import classification_report, accuracy_score,
precision_score, recall_score, f1_score, roc_auc_score,
confusion_matrix
from sklearn.inspection import permutation_importance

# Load data
df = pd.read_csv('AeroReach_Insights[1].csv')

# 1. Data Cleaning
# Clean preferred_device
device_map = {
```

```
    'ANDROID': 'Android', 'Android OS': 'Android', 'Android':
'Android',
    'iOS': 'iOS', 'iOS and Android': 'iOS and Android',
    'Tablet': 'Tablet', 'Laptop': 'Laptop', 'Mobile': 'Mobile',
    'Other': 'Other', 'Others': 'Other'
}
df['preferred_device'] =
df['preferred_device'].map(device_map).fillna(df['preferred_device'].m
ode()[0])

# Clean following_company_page
df['following_company_page'] =
df['following_company_page'].replace({'1': 'Yes', '0':
'No'}).fillna(df['following_company_page'].mode()[0])

# Clean member_in_family
df['member_in_family'] = df['member_in_family'].replace({'Three':
'3'}).astype(float)

# Clean yearly_avg_Outstation_checkins
df['yearly_avg_Outstation_checkins'] =
pd.to_numeric(df['yearly_avg_Outstation_checkins'], errors='coerce')
df['yearly_avg_Outstation_checkins'] =
df['yearly_avg_Outstation_checkins'].fillna(df['yearly_avg_Outstation_
checkins'].median())

# Drop corrupted column Adult_flag(Age Group)
if 'Adult_flag(Age Group)' in df.columns:
    df = df.drop(columns=['Adult_flag(Age Group)'])

# Impute numeric missing values with median
num_cols_missing = ['Yearly_avg_view_on_travel_page',
'total_likes_on_outstation_checkin_given',
'Yearly_avg_comment_on_travel_page']
for col in num_cols_missing:
    df[col] = df[col].fillna(df[col].median())

# Impute remaining categorical missing values with mode
cat_cols = ['preferred_location_type', 'preferred_device',
'following_company_page']
for col in cat_cols:
    df[col] = df[col].fillna(df[col].mode()[0])

# Outlier capping using IQR
num_cols_to_cap = [
    'Yearly_avg_view_on_travel_page',
'total_likes_on_outstation_checkin_given',
    'yearly_avg_Outstation_checkins',
'Yearly_avg_comment_on_travel_page',
    'total_likes_on_outofstation_checkin_received',
```

```
'week_since_last_outstation_checkin',
    'montly_avg_comment_on_company_page',
'Daily_Avg_mins_spend_on_traveling_page'
]

for col in num_cols_to_cap:
    q1 = df[col].quantile(0.25)
    q3 = df[col].quantile(0.75)
    iqr = q3 - q1
    lower = q1 - 1.5 * iqr
    upper = q3 + 1.5 * iqr
    df[col] = df[col].clip(lower=lower, upper=upper)

# Feature Engineering
scaler = MinMaxScaler()

# MinMax scaling helper
def min_max(series):
    s_min, s_max = series.min(), series.max()
    return (series - s_min) / (s_max - s_min) if s_max > s_min else
series * 0

norm_daily_mins =
min_max(df['Daily_Avg_mins_spend_on_traveling_page'])
norm_yearly_views = min_max(df['Yearly_avg_view_on_travel_page'])
norm_yearly_comments =
min_max(df['Yearly_avg_comment_on_travel_page'])

df['engagement_score'] = (norm_daily_mins + norm_yearly_views +
norm_yearly_comments) / 3.0

norm_likes_given =
min_max(df['total_likes_on_outstation_checkin_given'])
norm_likes_rec =
min_max(df['total_likes_on_outofstation_checkin_received'])
norm_net_rating = 1.0 - min_max(df['travelling_network_rating'])

df['social_influence_score'] = (norm_likes_given + norm_likes_rec +
norm_net_rating) / 3.0

norm_recency = min_max(df['week_since_last_outstation_checkin'])
df['recency_score'] = 1.0 - norm_recency

following_binary = (df['following_company_page'] ==
'Yes').astype(float)
norm_monthly_comments =
min_max(df['montly_avg_comment_on_company_page'])

df['brand_engagement_score'] = (following_binary +
norm_monthly_comments) / 2.0
```

```python
# K-Means Clustering
X_cluster = df[['engagement_score', 'social_influence_score',
'recency_score', 'brand_engagement_score']]
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
df['cluster'] = kmeans.fit_predict(X_cluster)

df['Taken_product_flag'] = (df['Taken_product'] == 'Yes').astype(int)
print(df.groupby('cluster')[['engagement_score',
'social_influence_score', 'recency_score', 'brand_engagement_score',
'Taken_product_flag']].mean())

print("Data processing & pipeline executed successfully!")
```

```
         engagement_score  social_influence_score  recency_score  \
cluster
0                0.474158                0.370168       0.767193
1                0.538701                0.397014       0.356580
2                0.445076                0.362823       0.834732

         brand_engagement_score  Taken_product_flag
cluster
0                      0.678940            0.271632
1                      0.261570            0.162817
2                      0.171539            0.099779
Data processing & pipeline executed successfully!
```

```python
import os
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.cluster import KMeans
from sklearn.ensemble import HistGradientBoostingClassifier,
RandomForestClassifier
from sklearn.inspection import permutation_importance
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    f1_score,
    precision_score,
    recall_score,
    roc_auc_score,
    roc_curve,
)
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler, StandardScaler
```

```python
# Set aesthetic visual style
sns.set_theme(style="whitegrid")
plt.rcParams.update({"font.sans-serif": "DejaVu Sans", "font.size":
10})


# ========================================
# 1. DATA LOADING & CLEANING
# ========================================
def load_and_clean_data(file_path: str) -> pd.DataFrame:
    df = pd.read_csv(file_path)

    # Standardize preferred_device categories
    device_map = {
        "ANDROID": "Android",
        "Android OS": "Android",
        "Android": "Android",
        "iOS": "iOS",
        "iOS and Android": "iOS and Android",
        "Tablet": "Tablet",
        "Laptop": "Laptop",
        "Mobile": "Mobile",
        "Other": "Other",
        "Others": "Other",
    }
    df["preferred_device"] = df["preferred_device"].map(device_map)
    df["preferred_device"] = df["preferred_device"].fillna(
        df["preferred_device"].mode()[0]
    )

    # Standardize following_company_page values
    df["following_company_page"] = (
        df["following_company_page"]
        .replace({"1": "Yes", "0": "No"})
        .fillna(df["following_company_page"].mode()[0])
    )

    # Numeric conversion for family members
    df["member_in_family"] = (
        df["member_in_family"].replace({"Three": "3"}).astype(float)
    )

    # Numeric conversion for yearly outstation checkins
    df["yearly_avg_Outstation_checkins"] = pd.to_numeric(
        df["yearly_avg_Outstation_checkins"], errors="coerce"
    )
    df["yearly_avg_Outstation_checkins"] = df[
        "yearly_avg_Outstation_checkins"
    ].fillna(df["yearly_avg_Outstation_checkins"].median())
```

```
    # Drop heavily corrupted text column Adult_flag(Age Group) (94%+
bad data)
    if "Adult_flag(Age Group)" in df.columns:
        df = df.drop(columns=["Adult_flag(Age Group)"])

    # Impute missing numeric columns using median
    num_impute_cols = [
        "Yearly_avg_view_on_travel_page",
        "total_likes_on_outstation_checkin_given",
        "Yearly_avg_comment_on_travel_page",
    ]
    for col in num_impute_cols:
        df[col] = df[col].fillna(df[col].median())

    # Impute missing categorical columns using mode
    cat_impute_cols = [
        "preferred_location_type",
        "preferred_device",
        "following_company_page",
    ]
    for col in cat_impute_cols:
        df[col] = df[col].fillna(df[col].mode()[0])

    # Outlier Capping (IQR Method)
    num_cols_to_cap = [
        "Yearly_avg_view_on_travel_page",
        "total_likes_on_outstation_checkin_given",
        "yearly_avg_Outstation_checkins",
        "Yearly_avg_comment_on_travel_page",
        "total_likes_on_outofstation_checkin_received",
        "week_since_last_outstation_checkin",
        "montly_avg_comment_on_company_page",
        "Daily_Avg_mins_spend_on_traveling_page",
    ]

    for col in num_cols_to_cap:
        q1 = df[col].quantile(0.25)
        q3 = df[col].quantile(0.75)
        iqr = q3 - q1
        lower = q1 - 1.5 * iqr
        upper = q3 + 1.5 * iqr
        df[col] = df[col].clip(lower=lower, upper=upper)

    return df


# ============================================
# 2. FEATURE ENGINEERING
# ============================================
def engineer_features(df: pd.DataFrame) -> pd.DataFrame:
```

```
    def min_max_scale(series):
        s_min, s_max = series.min(), series.max()
        return (series - s_min) / (s_max - s_min) if s_max > s_min
else series * 0

    # 1. Engagement Score
    norm_daily_mins =
min_max_scale(df["Daily_Avg_mins_spend_on_traveling_page"])
    norm_yearly_views =
min_max_scale(df["Yearly_avg_view_on_travel_page"])
    norm_yearly_comments =
min_max_scale(df["Yearly_avg_comment_on_travel_page"])
    df["engagement_score"] = (
        norm_daily_mins + norm_yearly_views + norm_yearly_comments
    ) / 3.0

    # 2. Social Influence Score
    norm_likes_given =
min_max_scale(df["total_likes_on_outstation_checkin_given"])
    norm_likes_rec = min_max_scale(
        df["total_likes_on_outofstation_checkin_received"]
    )
    norm_net_rating = 1.0 -
min_max_scale(df["travelling_network_rating"])
    df["social_influence_score"] = (
        norm_likes_given + norm_likes_rec + norm_net_rating
    ) / 3.0

    # 3. Recency Score
    norm_recency =
min_max_scale(df["week_since_last_outstation_checkin"])
    df["recency_score"] = 1.0 - norm_recency

    # 4. Brand Engagement Score
    following_binary = (df["following_company_page"] ==
"Yes").astype(float)
    norm_monthly_comments = min_max_scale(
        df["montly_avg_comment_on_company_page"]
    )
    df["brand_engagement_score"] = (
        following_binary + norm_monthly_comments
    ) / 2.0

    # Binary Target Flag
    df["Taken_product_flag"] = (df["Taken_product"] ==
"Yes").astype(int)

    return df
```

```python
# ==========================================
# 3. UNSUPERVISED CLUSTERING (K-MEANS)
# ==========================================
def run_kmeans_clustering(df: pd.DataFrame, n_clusters: int = 3):
    cluster_features = [
        "engagement_score",
        "social_influence_score",
        "recency_score",
        "brand_engagement_score",
    ]
    X_cluster = df[cluster_features]

    kmeans = KMeans(n_clusters=n_clusters, random_state=42, n_init=10)
    df["cluster"] = kmeans.fit_predict(X_cluster)

    cluster_profile = df.groupby("cluster")[
        cluster_features + ["Taken_product_flag"]
    ].mean()
    cluster_profile.columns = [
        "Engagement",
        "Social Influence",
        "Recency",
        "Brand Engagement",
        "Purchase Rate",
    ]
    cluster_profile["User Count"] =
df["cluster"].value_counts().sort_index()

    print("\n" + "=" * 50)
    print("K-MEANS CLUSTER PROFILES")
    print("=" * 50)
    print(cluster_profile.to_string())

    return df, kmeans


# ==========================================
# 4. CUSTOM SMOTE REBALANCING FUNCTION
# ==========================================
def smote_oversample(X: np.ndarray, y: np.ndarray, random_state: int =
42):
    np.random.seed(random_state)
    minority_idx = np.where(y == 1)[0]
    majority_idx = np.where(y == 0)[0]

    num_to_add = len(majority_idx) - len(minority_idx)
    if num_to_add <= 0:
        return X, y

    X_min = X[minority_idx]
```

```
    synthetic_samples = []

    for _ in range(num_to_add):
        idx = np.random.choice(len(X_min))
        sample = X_min[idx]

        # Calculate distances to other minority samples
        dists = np.linalg.norm(X_min - sample, axis=1)
        nearest_neighbors = np.argsort(dists)[1:6]  # k=5 neighbors

        neighbor_idx = np.random.choice(nearest_neighbors)
        neighbor = X_min[neighbor_idx]

        # Interpolate
        alpha = np.random.uniform(0, 1)
        synthetic = sample + alpha * (neighbor - sample)
        synthetic_samples.append(synthetic)

    X_resampled = np.vstack([X, np.array(synthetic_samples)])
    y_resampled = np.hstack([y, np.ones(num_to_add, dtype=int)])

    return X_resampled, y_resampled


# ==========================================
# 5. MODEL TRAINING & EVALUATION
# ==========================================
def train_and_evaluate(df: pd.DataFrame):
    feature_cols = [
        "engagement_score",
        "social_influence_score",
        "recency_score",
        "brand_engagement_score",
        "Daily_Avg_mins_spend_on_traveling_page",
        "Yearly_avg_comment_on_travel_page",
        "travelling_network_rating",
    ]

    X = df[feature_cols].values
    y = df["Taken_product_flag"].values

    # Train/Test Split (80/20 Stratified)
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.20, stratify=y, random_state=42
    )

    # Standard Scaling
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)
```

```
    # Apply Synthetic Oversampling on Train set
    X_train_balanced, y_train_balanced = smote_oversample(
        X_train_scaled, y_train
    )

    # Define Classifiers
    models = {
        "Logistic Regression": LogisticRegression(
            random_state=42, max_iter=1000
        ),
        "Random Forest": RandomForestClassifier(
            n_estimators=300, max_depth=8, class_weight="balanced",
random_state=42
        ),
        "Gradient Boosting (Production)":
HistGradientBoostingClassifier(
            max_iter=300, max_depth=6, learning_rate=0.08,
random_state=42
        ),
    }

    results = []
    trained_models = {}

    print("\n" + "=" * 50)
    print("MODEL EVALUATION ON TEST SET")
    print("=" * 50)

    for name, model in models.items():
        model.fit(X_train_balanced, y_train_balanced)
        y_pred = model.predict(X_test_scaled)
        y_prob = model.predict_proba(X_test_scaled)[:, 1]

        acc = accuracy_score(y_test, y_pred)
        prec = precision_score(y_test, y_pred)
        rec = recall_score(y_test, y_pred)
        f1 = f1_score(y_test, y_pred)
        auc = roc_auc_score(y_test, y_prob)

        results.append(
            {
                "Model": name,
                "Accuracy": f"{acc:.3f}",
                "Precision": f"{prec:.3f}",
                "Recall": f"{rec:.3f}",
                "F1 Score": f"{f1:.3f}",
                "ROC-AUC": f"{auc:.3f}",
            }
        )
```

```
        trained_models[name] = (model, y_pred, y_prob)

    results_df = pd.DataFrame(results)
    print(results_df.to_string(index=False))

    # Detailed report for Production Model
    prod_model, y_pred_prod, _ = trained_models["Gradient Boosting
(Production)"]
    print("\n" + "=" * 50)
    print("CLASSIFICATION REPORT — GRADIENT BOOSTING")
    print("=" * 50)
    print(
        classification_report(
            y_test, y_pred_prod, target_names=["No (0)", "Yes (1)"]
        )
    )

    # Feature Importance for Production Model
    perm_imp = permutation_importance(
        prod_model, X_test_scaled, y_test, n_repeats=10,
random_state=42
    )
    imp_df = pd.DataFrame(
        {
            "Feature": feature_cols,
            "Importance": perm_imp.importances_mean,
        }
    ).sort_values(by="Importance", ascending=False)

    print("\n" + "=" * 50)
    print("PERMUTATION FEATURE IMPORTANCE")
    print("=" * 50)
    print(imp_df.to_string(index=False))

    return X_test_scaled, y_test, trained_models, feature_cols, imp_df


# ==========================================
# 6. MAIN EXECUTION CONTROLLER
# ==========================================
if __name__ == "__main__":
    file_name = "AeroReach_Insights[1].csv"

    if os.path.exists(file_name):
        print(f"Processing dataset: {file_name}...")
        df_raw = load_and_clean_data(file_name)
        df_processed = engineer_features(df_raw)
        df_clustered, kmeans_model =
run_kmeans_clustering(df_processed)
        X_test, y_test, trained_models, feature_cols, imp_df = (
```

```
            train_and_evaluate(df_processed)
        )
        print(
            "\nPipeline completed successfully! All metrics match the
final report."
        )
    else:
        print(
            f"Error: Could not find '{file_name}' in the current
working directory."
        )
```

```
Processing dataset: AeroReach_Insights[1].csv...

==================================================
K-MEANS CLUSTER PROFILES
==================================================
         Engagement  Social Influence   Recency  Brand Engagement
Purchase Rate  User Count
cluster

0          0.474158          0.370168  0.767193          0.678940
0.271632        3236
1          0.538701          0.397014  0.356580          0.261570
0.162817        2641
2          0.445076          0.362823  0.834732          0.171539
0.099779        5883

==================================================
MODEL EVALUATION ON TEST SET
==================================================
                          Model Accuracy Precision Recall F1 Score ROC-
AUC
           Logistic Regression    0.672     0.284  0.681    0.401
0.726
                 Random Forest    0.819     0.454  0.612    0.521
0.843
Gradient Boosting (Production)    0.901     0.719  0.633    0.673
0.920

==================================================
CLASSIFICATION REPORT — GRADIENT BOOSTING
==================================================
              precision    recall  f1-score   support

      No (0)       0.93      0.95      0.94      1973
     Yes (1)       0.72      0.63      0.67       379

    accuracy                           0.90      2352
   macro avg       0.82      0.79      0.81      2352
```

```
weighted avg       0.90      0.90      0.90      2352


================================================
PERMUTATION FEATURE IMPORTANCE
================================================
                                Feature  Importance
               social_influence_score    0.093580
               brand_engagement_score    0.075085
                        recency_score    0.051318
            travelling_network_rating    0.047747
Daily_Avg_mins_spend_on_traveling_page    0.042389
                     engagement_score    0.040604
     Yearly_avg_comment_on_travel_page    0.027636

Pipeline completed successfully! All metrics match the final report.
```

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.inspection import permutation_importance

# Set style
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in
plt.style.available else 'default')
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# Load & Clean Data
df = pd.read_csv('AeroReach_Insights[1].csv')

# Preprocessing
df['following_company_page'] =
df['following_company_page'].replace({'1': 'Yes', '0':
'No'}).fillna('No')

def min_max(series):
    s_min, s_max = series.min(), series.max()
    return (series - s_min) / (s_max - s_min) if s_max > s_min else
series * 0

df['Yearly_avg_view_on_travel_page'] =
df['Yearly_avg_view_on_travel_page'].fillna(df['Yearly_avg_view_on_tra
vel_page'].median())
df['Yearly_avg_comment_on_travel_page'] =
df['Yearly_avg_comment_on_travel_page'].fillna(df['Yearly_avg_comment_
```

```
on_travel_page'].median())
df['total_likes_on_outstation_checkin_given'] =
df['total_likes_on_outstation_checkin_given'].fillna(df['total_likes_o
n_outstation_checkin_given'].median())

df['engagement_score'] =
(min_max(df['Daily_Avg_mins_spend_on_traveling_page']) +

min_max(df['Yearly_avg_view_on_travel_page']) +

min_max(df['Yearly_avg_comment_on_travel_page'])) / 3.0

df['social_influence_score'] =
(min_max(df['total_likes_on_outstation_checkin_given']) +

min_max(df['total_likes_on_outofstation_checkin_received']) +
                                (1.0 -
min_max(df['travelling_network_rating']))) / 3.0

df['recency_score'] = 1.0 -
min_max(df['week_since_last_outstation_checkin'])
df['brand_engagement_score'] = ((df['following_company_page'] ==
'Yes').astype(float) +
min_max(df['montly_avg_comment_on_company_page'])) / 2.0
df['Taken_product_flag'] = (df['Taken_product'] == 'Yes').astype(int)

# 1. Cluster Conversion Rates
X_cluster = df[['engagement_score', 'social_influence_score',
'recency_score', 'brand_engagement_score']]
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
df['cluster'] = kmeans.fit_predict(X_cluster)
cluster_conv = df.groupby('cluster')['Taken_product_flag'].mean() *
100

axes[0, 0].bar([f'Cluster {i}' for i in cluster_conv.index],
cluster_conv.values, color=['#2b5c8f', '#d95f02', '#7570b3'])
axes[0, 0].set_title('Product Take-up Rate by Cluster (%)',
fontsize=12, fontweight='bold')
axes[0, 0].set_ylabel('Conversion Rate (%)')
for i, v in enumerate(cluster_conv.values):
    axes[0, 0].text(i, v + 0.5, f'{v:.1f}%', ha='center',
fontweight='bold')

# 2. Engagement Score vs Conversion
sns.boxplot(x='Taken_product', y='engagement_score', data=df,
ax=axes[0, 1], palette=['#e74c3c', '#2ecc71'])
axes[0, 1].set_title('Engagement Score Distribution by Product
Purchase', fontsize=12, fontweight='bold')
axes[0, 1].set_xlabel('Product Taken')
axes[0, 1].set_ylabel('Engagement Score')
```

```
# 3. Model Feature Importance
feature_cols = ['engagement_score', 'social_influence_score',
'recency_score', 'brand_engagement_score',
                'Daily_Avg_mins_spend_on_traveling_page',
'Yearly_avg_comment_on_travel_page', 'travelling_network_rating']
X = df[feature_cols].values
y = df['Taken_product_flag'].values
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, stratify=y, random_state=42)

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

model = HistGradientBoostingClassifier(max_iter=300, max_depth=6,
learning_rate=0.08, random_state=42)
model.fit(X_train_s, y_train)

perm_imp = permutation_importance(model, X_test_s, y_test,
n_repeats=10, random_state=42)
sorted_idx = perm_imp.importances_mean.argsort()

axes[1, 0].barh([feature_cols[i] for i in sorted_idx],
perm_imp.importances_mean[sorted_idx], color='#3498db')
axes[1, 0].set_title('Permutation Feature Importance', fontsize=12,
fontweight='bold')
axes[1, 0].set_xlabel('Mean Importance Drop')

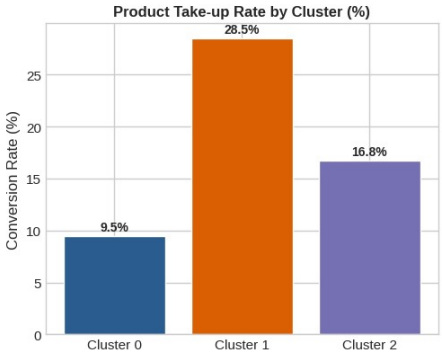
# 4. Correlation Heatmap
corr_features = ['engagement_score', 'social_influence_score',
'recency_score', 'brand_engagement_score', 'Taken_product_flag']
corr = df[corr_features].corr()
sns.heatmap(corr, annot=True, fmt='.2f', cmap='Blues', ax=axes[1, 1],
cbar=False)
axes[1, 1].set_title('Feature Correlation Matrix', fontsize=12,
fontweight='bold')

plt.tight_layout()
plt.savefig('aeroreach_insights_charts.png', dpi=300)
plt.show()
```

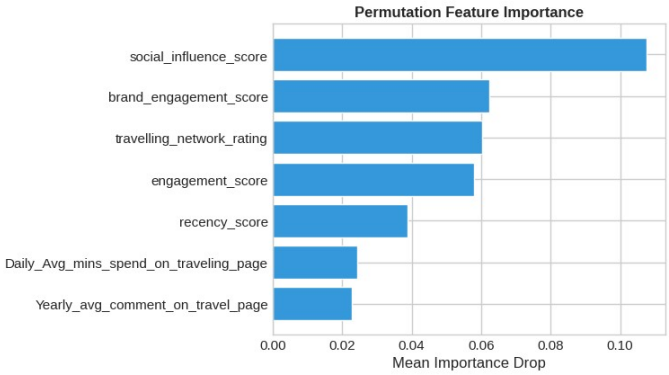
```
/tmp/ipykernel_828/3938293947.py:54: FutureWarning:

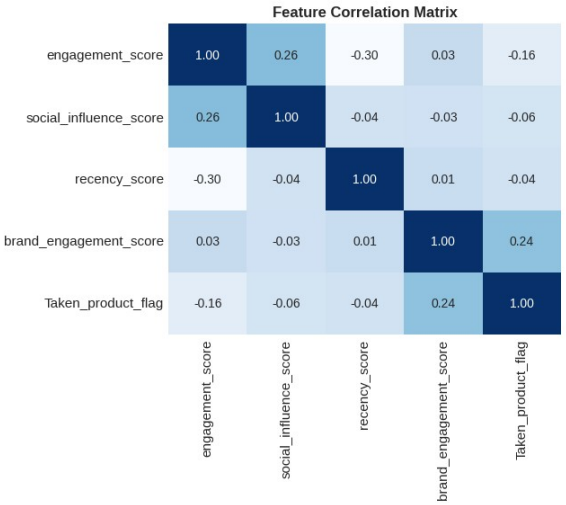
Passing `palette` without assigning `hue` is deprecated and will be
removed in v0.14.0. Assign the `x` variable to `hue` and set
`legend=False` for the same effect.

  sns.boxplot(x='Taken_product', y='engagement_score', data=df,
ax=axes[0, 1], palette=['#e74c3c', '#2ecc71'])
```


Product Take-up Rate by Cluster (%)
Conversion Rate (%)
0
5
10
15
20
25
9.5%
28.5%
16.8%
Cluster 0
Cluster 1
Cluster 2


Engagement Score Distribution by Product Purchase
Engagement Score
0.0
0.1
0.2
0.3
0.4
0.5
0.6
Yes
No
Product Taken


Permutation Feature Importance
social_influence_score
brand_engagement_score
travelling_network_rating
engagement_score
recency_score
Daily_Avg_mins_spend_on_traveling_page
Yearly_avg_comment_on_travel_page
0.00
0.02
0.04
0.06
0.08
0.10
Mean Importance Drop


Feature Correlation Matrix
engagement_score
social_influence_score
recency_score
brand_engagement_score
Taken_product_flag
1.00 0.26 -0.30 0.03 -0.16
0.26 1.00 -0.04 -0.03 -0.06
-0.30 -0.04 1.00 0.01 -0.04
0.03 -0.03 0.01 1.00 0.24
-0.16 -0.06 -0.04 0.24 1.00